

CO4_AT3_INDUSTRY TASK

1.EMPLOYEE PERFORMANCE & PAYROLL DASHBOARD :

JAVA CODE :-

```
import java.awt.*;
import java.awt.event.*;

public class EmployeePayroll extends Frame implements ActionListener {

    Label title, l1, l2, l3, l4, l5, l6, l7;

    TextField idField, nameField, salaryField, daysField;

    Choice department, designation;

    Checkbox excellent, good, average, poor;

    CheckboxGroup ratingGroup;

    Button generate, reset;

    TextArea report;

    EmployeePayroll() {

        setTitle("Employee Performance & Payroll Dashboard");

        setSize(750, 650);

        setLayout(new BorderLayout(10, 10));

        title = new Label("EMPLOYEE PERFORMANCE & PAYROLL DASHBOARD",
Label.CENTER);

        title.setFont(new Font("Arial", Font.BOLD, 20));

        title.setBackground(Color.DARK_GRAY);

        title.setForeground(Color.WHITE);

        add(title, BorderLayout.NORTH);

        Panel form = new Panel();

        form.setLayout(new GridBagLayout());

        GridBagConstraints gbc = new GridBagConstraints();

        gbc.insets = new Insets(8, 8, 8, 8);

        gbc.fill = GridBagConstraints.HORIZONTAL;
```

```
l1 = new Label("Employee ID");
idField = new TextField(20);
l2 = new Label("Employee Name");
nameField = new TextField(20);
l3 = new Label("Department");
department = new Choice();
department.add("IT");
department.add("HR");
department.add("Finance");
department.add("Marketing");
department.add("Operations");
l4 = new Label("Designation");
designation = new Choice();
designation.add("Developer");
designation.add("Manager");
designation.add("Tester");
designation.add("Analyst");
designation.add("Team Lead");
l5 = new Label("Basic Salary");
salaryField = new TextField(20);
l6 = new Label("Working Days");
daysField = new TextField(20);
l7 = new Label("Performance Rating");
ratingGroup = new CheckboxGroup();
excellent = new Checkbox("Excellent", ratingGroup, true);
good = new Checkbox("Good", ratingGroup, false);
average = new Checkbox("Average", ratingGroup, false);
poor = new Checkbox("Poor", ratingGroup, false);
Panel ratingPanel = new Panel(new FlowLayout(FlowLayout.LEFT, 5, 0));
```

```
ratingPanel.add(excellent);
ratingPanel.add(good);
ratingPanel.add(average);
ratingPanel.add(poor);
gbc.gridx = 0;
gbc.gridy = 0;
form.add(l1, gbc);
gbc.gridx = 1;
form.add(idField, gbc);
gbc.gridx = 0;
gbc.gridy = 1;
form.add(l2, gbc);
gbc.gridx = 1;
form.add(nameField, gbc);
gbc.gridx = 0;
gbc.gridy = 2;
form.add(l3, gbc);
gbc.gridx = 1;
form.add(department, gbc);
gbc.gridx = 0;
gbc.gridy = 3;
form.add(l4, gbc);
gbc.gridx = 1;
form.add(designation, gbc);
gbc.gridx = 0;
gbc.gridy = 4;
form.add(l5, gbc);
gbc.gridx = 1;
form.add(salaryField, gbc);
```

```
gbc.gridx = 0;
gbc.gridy = 5;
form.add(l6, gbc);
gbc.gridx = 1;
form.add(daysField, gbc);
gbc.gridx = 0;
gbc.gridy = 6;
form.add(l7, gbc);
gbc.gridx = 1;
form.add(ratingPanel, gbc);
generate = new Button("GENERATE REPORT");
reset = new Button("RESET");
generate.setBackground(Color.GREEN);
reset.setBackground(Color.LIGHT_GRAY);
Panel buttonPanel = new Panel(new FlowLayout());
buttonPanel.add(generate);
buttonPanel.add(reset);
gbc.gridx = 0;
gbc.gridy = 7;
gbc.gridwidth = 2;
form.add(buttonPanel, gbc);
add(form, BorderLayout.CENTER);
report = new TextArea(12, 70);
report.setEditable(false);
report.setFont(new Font("Monospaced", Font.PLAIN, 14));
Panel reportPanel = new Panel(new BorderLayout());
Label reportTitle = new Label("PAYROLL REPORT", Label.CENTER);
reportTitle.setFont(new Font("Arial", Font.BOLD, 16));
reportPanel.add(reportTitle, BorderLayout.NORTH);
```

```

reportPanel.add(report, BorderLayout.CENTER);
add(reportPanel, BorderLayout.SOUTH);
generate.addActionListener(this);
reset.addActionListener(this);
addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        dispose();
    }
});
setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    if (e.getSource() == generate) {
        String id = idField.getText().trim();
        String name = nameField.getText().trim();
        String salaryText = salaryField.getText().trim();
        String daysText = daysField.getText().trim();
        if (id.equals("") || name.equals("") ||
            salaryText.equals("") || daysText.equals("")) {
            report.setText("ERROR\n\nPlease enter all required employee details.");
            return;
        }
        try {
            double basicSalary = Double.parseDouble(salaryText);
            int workingDays = Integer.parseInt(daysText);
            if (basicSalary <= 0) {
                report.setText("ERROR\n\nBasic salary must be greater than zero.");
                return;
            }
        }
    }
}

```

```

if (workingDays <= 0 || workingDays > 26) {
    report.setText("ERROR\n\nWorking days must be between 1 and 26.");
    return;
}

double adjustedSalary;
double bonusRate;
double performanceBonus;
double grossSalary;
double netSalary;
adjustedSalary = basicSalary * workingDays / 26.0;
if (excellent.getState()) {
    bonusRate = 0.15;
} else if (good.getState()) {
    bonusRate = 0.10;
} else if (average.getState()) {
    bonusRate = 0.05;
} else {
    bonusRate = 0.00;
}

performanceBonus = adjustedSalary * bonusRate;
grossSalary = adjustedSalary + performanceBonus;
netSalary = grossSalary;

String rating =
    ratingGroup.getSelectedCheckbox().getLabel();
report.setText("");

report.append("=====\n");
report.append("      EMPLOYEE PAYROLL REPORT\n");
report.append("=====\n");
report.append("Employee ID      : " + id + "\n");

```

```

report.append("Employee Name      : " + name + "\n");
report.append("Department        : " +
              department.getSelectedItemAt() + "\n");
report.append("Designation       : " +
              designation.getSelectedItemAt() + "\n");
report.append("Basic Salary      : " +
              String.format("%.2f", basicSalary) + "\n");
report.append("Working Days      : " + workingDays + "\n");
report.append("Performance Rating : " + rating + "\n");
report.append("-----\n");
report.append("Adjusted Salary   : " +
              String.format("%.2f", adjustedSalary) + "\n");
report.append("Performance Bonus : " +
              String.format("%.2f", performanceBonus) + "\n");
report.append("Gross Salary      : " +
              String.format("%.2f", grossSalary) + "\n");
report.append("Net Salary       : " +
              String.format("%.2f", netSalary) + "\n");
report.append("=====\n");
} catch (NumberFormatException ex) {
    report.setText(
        "ERROR\n\nBasic Salary and Working Days must be numeric values."
    );
}
}
else if (e.getSource() == reset) {
    idField.setText("");
    nameField.setText("");
    salaryField.setText("");
}

```

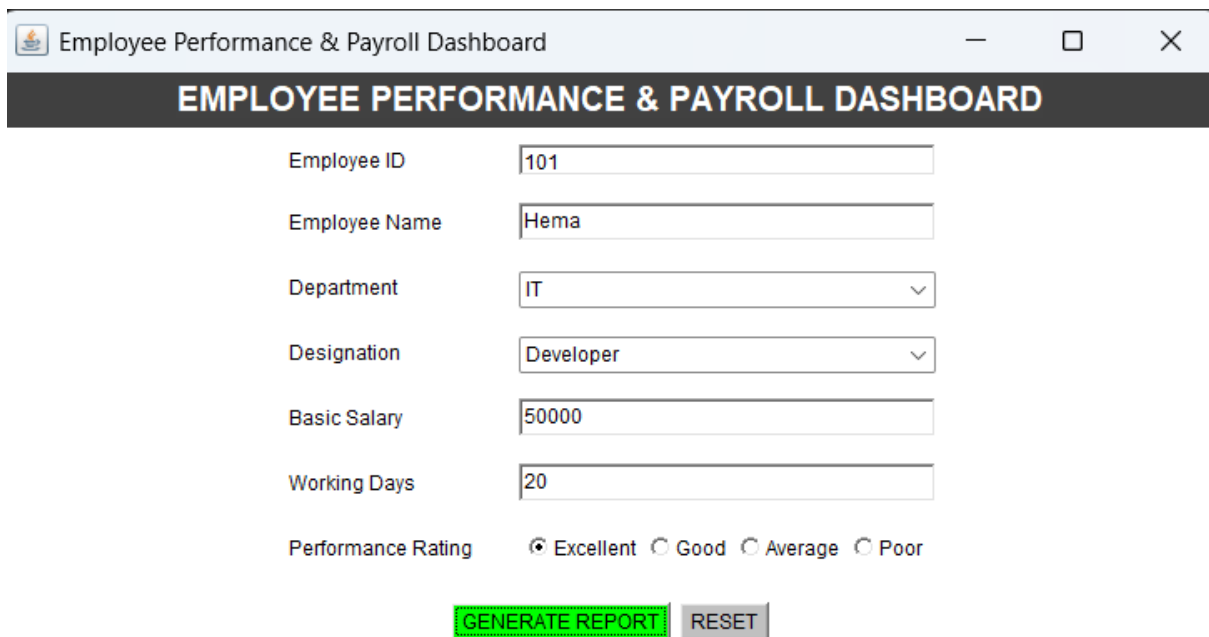
```

        daysField.setText("");
        department.select(0);
        designation.select(0);
        excellent.setState(true);
        report.setText("");
    }
}

public static void main(String[] args) {
    new EmployeePayroll();
}
}

```

OUTPUT :



The screenshot shows a Java Swing window titled "Employee Performance & Payroll Dashboard". The window has a dark header bar with the title in white. Below the header, the form contains the following fields and controls:

- Employee ID:** A text input field containing the value "101".
- Employee Name:** A text input field containing the value "Hema".
- Department:** A dropdown menu with "IT" selected.
- Designation:** A dropdown menu with "Developer" selected.
- Basic Salary:** A text input field containing the value "50000".
- Working Days:** A text input field containing the value "20".
- Performance Rating:** A group of four radio buttons labeled "Excellent", "Good", "Average", and "Poor". The "Excellent" radio button is selected.
- Buttons:** At the bottom, there are two buttons: "GENERATE REPORT" (highlighted in green) and "RESET" (greyed out).

PAYROLL REPORT

EMPLOYEE PAYROLL REPORT	
Employee ID	: 101
Employee Name	: Hema
Department	: IT
Designation	: Developer
Basic Salary	: 50000.00
Working Days	: 20
Performance Rating	: Excellent

Adjusted Salary	: 38461.54
Performance Bonus	: 5769.23

PAYROLL REPORT

Employee ID	: 101
Employee Name	: Hema
Department	: IT
Designation	: Developer
Basic Salary	: 50000.00
Working Days	: 20
Performance Rating	: Excellent

Adjusted Salary	: 38461.54
Performance Bonus	: 5769.23
Gross Salary	: 44230.77
Net Salary	: 44230.77

2.ONLINE FOOD ORDERING AND BILLING SYSTEM :

JAVA CODE :

```
import java.awt.*;
import java.awt.event.*;

public class FoodOrderingBilling extends Frame implements ActionListener {

    TextField customerField;

    Choice item1, item2, item3;

    TextField qty1, qty2, qty3;

    Button orderButton, resetButton;

    TextArea bill;

    Color cream = new Color(255, 248, 230);
    Color red = new Color(180, 45, 35);
    Color orange = new Color(245, 150, 40);
    Color green = new Color(45, 145, 70);
```

```
FoodCanvas foodCanvas;

FoodOrderingBilling() {
    setTitle("Foodie Express - Online Food Ordering");
    setSize(1000, 800);
    setLayout(new BorderLayout(10, 10));
    setBackground(cream);

    Panel header = new Panel(new BorderLayout());
    header.setBackground(red);

    Label title = new Label("FOODIE EXPRESS", Label.CENTER);
    title.setFont(new Font("Arial", Font.BOLD, 30));
    title.setForeground(Color.WHITE);

    Label subtitle = new Label(
        "ONLINE FOOD ORDERING & BILLING SYSTEM",
        Label.CENTER
    );
    subtitle.setFont(new Font("Arial", Font.BOLD, 14));
    subtitle.setForeground(Color.WHITE);

    header.add(title, BorderLayout.CENTER);
    header.add(subtitle, BorderLayout.SOUTH);
    add(header, BorderLayout.NORTH);

    Panel center = new Panel(new BorderLayout());
    foodCanvas = new FoodCanvas();
    center.add(foodCanvas, BorderLayout.NORTH);

    Panel orderPanel = new Panel(new GridBagLayout());
    orderPanel.setBackground(cream);

    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(6, 10, 6, 10);
    gbc.fill = GridBagConstraints.HORIZONTAL;

    Label customerLabel = new Label("Customer Name");
```

```
customerLabel.setFont(new Font("Arial", Font.BOLD, 15));
customerField = new TextField(25);
gbc.gridx = 0;
gbc.gridy = 0;
orderPanel.add(customerLabel, gbc);
gbc.gridx = 1;
gbc.gridwidth = 3;
orderPanel.add(customerField, gbc);
gbc.gridwidth = 1;
Label food1 = new Label("Food Item 1");
food1.setFont(new Font("Arial", Font.BOLD, 14));
item1 = createChoice();
qty1 = new TextField(5);
gbc.gridx = 0;
gbc.gridy = 1;
orderPanel.add(food1, gbc);
gbc.gridx = 1;
orderPanel.add(item1, gbc);
gbc.gridx = 2;
orderPanel.add(new Label("Quantity"), gbc);
gbc.gridx = 3;
orderPanel.add(qty1, gbc);
Label food2 = new Label("Food Item 2");
food2.setFont(new Font("Arial", Font.BOLD, 14));
item2 = createChoice();
qty2 = new TextField(5);
gbc.gridx = 0;
gbc.gridy = 2;
orderPanel.add(food2, gbc);
```

```
gbc.gridx = 1;
orderPanel.add(item2, gbc);
gbc.gridx = 2;
orderPanel.add(new Label("Quantity"), gbc);
gbc.gridx = 3;
orderPanel.add(qty2, gbc);
Label food3 = new Label("Food Item 3");
food3.setFont(new Font("Arial", Font.BOLD, 14));
item3 = createChoice();
qty3 = new TextField(5);
gbc.gridx = 0;
gbc.gridy = 3;
orderPanel.add(food3, gbc);
gbc.gridx = 1;
orderPanel.add(item3, gbc);
gbc.gridx = 2;
orderPanel.add(new Label("Quantity"), gbc);
gbc.gridx = 3;
orderPanel.add(qty3, gbc);
orderButton = new Button("PLACE ORDER");
orderButton.setFont(new Font("Arial", Font.BOLD, 15));
orderButton.setBackground(green);
orderButton.setForeground(Color.WHITE);
resetButton = new Button("RESET");
resetButton.setFont(new Font("Arial", Font.BOLD, 15));
resetButton.setBackground(red);
resetButton.setForeground(Color.WHITE);
Panel buttons = new Panel();
buttons.setBackground(cream);
```

```

buttons.add(orderButton);

buttons.add(resetButton);

gbc.gridx = 0;

gbc.gridy = 4;

gbc.gridwidth = 4;

orderPanel.add(buttons, gbc);

center.add(orderPanel, BorderLayout.CENTER);

add(center, BorderLayout.CENTER);

Panel billPanel = new Panel(new BorderLayout());

Label billTitle = new Label("YOUR BILL", Label.CENTER);

billTitle.setFont(new Font("Arial", Font.BOLD, 20));

billTitle.setForeground(red);

bill = new TextArea(9, 90);

bill.setFont(new Font("Monospaced", Font.PLAIN, 14));

bill.setEditable(false);

bill.setBackground(Color.WHITE);

billPanel.add(billTitle, BorderLayout.NORTH);

billPanel.add(bill, BorderLayout.CENTER);

add(billPanel, BorderLayout.SOUTH);

orderButton.addActionListener(this);

resetButton.addActionListener(this);

addWindowListener(new WindowAdapter() {

    public void windowClosing(WindowEvent e) {

        dispose();

    }

});

setVisible(true);

}

Choice createChoice() {

```

```
Choice c = new Choice();
c.add("Pizza - Rs.250");
c.add("Burger - Rs.150");
c.add("Sandwich - Rs.100");
c.add("Biryani - Rs.220");
c.add("Fried Rice - Rs.180");
return c;
}

double getPrice(String item) {
    if (item.startsWith("Pizza"))
        return 250;
    if (item.startsWith("Burger"))
        return 150;
    if (item.startsWith("Sandwich"))
        return 100;
    if (item.startsWith("Biryani"))
        return 220;
    return 180;
}

String getFoodName(String item) {
    if (item.startsWith("Pizza"))
        return "Pizza";
    if (item.startsWith("Burger"))
        return "Burger";
    if (item.startsWith("Sandwich"))
        return "Sandwich";
    if (item.startsWith("Biryani"))
        return "Biryani";
    return "Fried Rice";
}
```

```

}

public void actionPerformed(ActionEvent e) {
    if (e.getSource() == orderButton) {
        String customer = customerField.getText().trim();
        if (customer.equals("")) {
            bill.setText(
                "\n ERROR: Please enter customer name."
            );
            return;
        }
        try {
            int n1 = Integer.parseInt(qty1.getText().trim());
            int n2 = Integer.parseInt(qty2.getText().trim());
            int n3 = Integer.parseInt(qty3.getText().trim());
            if (n1 < 0 || n2 < 0 || n3 < 0) {
                bill.setText(
                    "\n ERROR: Quantity cannot be negative."
                );
                return;
            }
            String food1 = getFoodName(item1.getSelectedItemAt());
            String food2 = getFoodName(item2.getSelectedItemAt());
            String food3 = getFoodName(item3.getSelectedItemAt());
            double price1 = getPrice(item1.getSelectedItemAt());
            double price2 = getPrice(item2.getSelectedItemAt());
            double price3 = getPrice(item3.getSelectedItemAt());
            double amount1 = price1 * n1;
            double amount2 = price2 * n2;
            double amount3 = price3 * n3;

```

```

double subtotal = amount1 + amount2 + amount3;
if (subtotal == 0) {
    bill.setText(
        "\n  ERROR: Please order at least one item."
    );
    return;
}
double discount = 0;
if (subtotal >= 1000) {
    discount = subtotal * 0.10;
}
double taxableAmount = subtotal - discount;
double tax = taxableAmount * 0.05;
double total = taxableAmount + tax;
bill.setText("");
bill.append(
    "===== \n"
);
bill.append(
    "          FOODIE EXPRESS \n"
);
bill.append(
    "          FINAL BILL \n"
);
bill.append(
    "===== \n"
);

```



```

bill.append(
    "Customer Name : " + customer + "\n"
);
bill.append(
    "-----\n"
);
bill.append(
    String.format(
        "%-18s %-10s %-12s %-12s\n",
        "ITEM", "QTY", "PRICE", "AMOUNT"
    )
);
bill.append(
    "-----\n"
);
if (n1 > 0) {
    bill.append(
        String.format(
            "%-18s %-10d %-12.2f %-12.2f\n",
            food1, n1, price1, amount1
        )
    );
}
if (n2 > 0) {
    bill.append(
        String.format(
            "%-18s %-10d %-12.2f %-12.2f\n",
            food2, n2, price2, amount2
        )
    );
}

```

```

    );
}
if (n3 > 0) {
    bill.append(
        String.format(
            "%-18s %-10d %-12.2f %-12.2f\n",
            food3, n3, price3, amount3
        )
    );
}
bill.append(
    "-----\n"
);
bill.append(
    String.format(
        "Subtotal      : Rs. %.2f\n",
        subtotal
    )
);
bill.append(
    String.format(
        "Discount      : Rs. %.2f\n",
        discount
    )
);
bill.append(
    String.format(
        "Tax (5%%)      : Rs. %.2f\n",
        tax

```

```

        )
    );
    bill.append(
        String.format(
            "GRAND TOTAL      : Rs. %.2f\n",
            total
        )
    );
    bill.append(
        "===== \n"

    );
    bill.append(
        "          THANK YOU! VISIT AGAIN!\n"

    );
    bill.append(
        "===== \n"

    );
    } catch (NumberFormatException ex) {
        bill.setText(
            "\n  ERROR: Please enter valid numeric quantities."
        );
    }
}

if (e.getSource() == resetButton) {
    customerField.setText("");
    qty1.setText("");
    qty2.setText("");
    qty3.setText("");
    item1.select(0);

```

```

        item2.select(0);
        item3.select(0);
        bill.setText("");
    }
}

class FoodCanvas extends Canvas {
    FoodCanvas() {
        setBackground(cream);
        setSize(950, 310);
    }

    public void paint(Graphics g) {
        int w = getWidth();
        g.setColor(new Color(255, 225, 170));
        g.fillRoundRect(15, 10, w - 30, 285, 30, 30);
        drawFoodCard(g, 40, "PIZZA", "Rs.250", 0);
        drawFoodCard(g, 350, "BURGER", "Rs.150", 1);
        drawFoodCard(g, 660, "SANDWICH", "Rs.100", 2);
    }

    void drawFoodCard(Graphics g, int x, String name,
        String price, int type) {
        g.setColor(Color.WHITE);
        g.fillRoundRect(x, 30, 250, 245, 25, 25);
        g.setColor(new Color(220, 60, 40));
        g.drawRoundRect(x, 30, 250, 245, 25, 25);
        if (type == 0)
            drawPizza(g, x + 125, 135);
        if (type == 1)
            drawBurger(g, x + 125, 135);
        if (type == 2)

```

```

        drawSandwich(g, x + 125, 135);
    g.setColor(Color.DARK_GRAY);
    g.setFont(new Font("Arial", Font.BOLD, 18));
    g.drawString(name, x + 85, 235);
    g.setFont(new Font("Arial", Font.BOLD, 15));
    g.setColor(new Color(180, 45, 35));
    g.drawString(price, x + 100, 258);
}

void drawPizza(Graphics g, int x, int y) {
    g.setColor(new Color(210, 130, 50));
    g.fillOval(x - 82, y - 82, 164, 164);
    g.setColor(new Color(255, 205, 65));
    g.fillOval(x - 72, y - 72, 144, 144);
    g.setColor(new Color(245, 190, 45));
    g.fillArc(x - 70, y - 70, 140, 140, 25, 60);
    g.setColor(new Color(190, 50, 40));
    g.fillOval(x - 45, y - 35, 22, 22);
    g.fillOval(x + 15, y - 45, 22, 22);
    g.fillOval(x + 30, y + 20, 22, 22);
    g.fillOval(x - 35, y + 25, 22, 22);
    g.fillOval(x - 5, y + 5, 22, 22);
    g.setColor(new Color(80, 150, 55));
    g.fillOval(x - 20, y - 45, 13, 13);
    g.fillOval(x + 40, y - 5, 13, 13);
    g.fillOval(x - 50, y + 5, 13, 13);
    g.setColor(new Color(150, 85, 30));
    g.drawOval(x - 82, y - 82, 164, 164);
}

void drawBurger(Graphics g, int x, int y) {

```

```
g.setColor(new Color(220, 145, 55));
g.fillArc(x - 90, y - 75, 180, 100, 0, 180);
g.setColor(new Color(245, 175, 75));
g.fillArc(x - 82, y - 68, 164, 85, 0, 180);
g.setColor(new Color(255, 235, 150));
g.fillOval(x - 45, y - 45, 8, 5);
g.fillOval(x - 10, y - 52, 8, 5);
g.fillOval(x + 30, y - 43, 8, 5);
g.fillOval(x + 55, y - 25, 8, 5);
g.setColor(new Color(75, 160, 65));
g.fillRect(x - 80, y + 5, 160, 18);
g.setColor(new Color(255, 205, 40));
int[] cheeseX = {
    x - 75, x + 75, x + 60, x + 30,
    x + 5, x - 25, x - 55, x - 75
};
int[] cheeseY = {
    y + 18, y + 18, y + 38, y + 28,
    y + 40, y + 27, y + 38, y + 18
};
g.fillPolygon(cheeseX, cheeseY, 8);
g.setColor(new Color(95, 50, 25));
g.fillRoundRect(x - 78, y + 35, 156, 38, 10, 10);
g.setColor(new Color(220, 55, 40));
g.fillOval(x - 50, y + 38, 22, 10);
g.fillOval(x + 20, y + 38, 22, 10);
g.setColor(new Color(220, 145, 55));
g.fillRoundRect(x - 80, y + 72, 160, 35, 12, 12);
g.setColor(new Color(150, 85, 25));
```

```

    g.drawRoundRect(x - 90, y - 75, 180, 182, 20, 20);
}

void drawSandwich(Graphics g, int x, int y) {
    int[] bread1X = {
        x - 80, x + 80, x + 10
    };
    int[] bread1Y = {
        y - 70, y - 70, y + 5
    };
    g.setColor(new Color(240, 195, 105));
    g.fillPolygon(bread1X, bread1Y, 3);
    g.setColor(new Color(255, 220, 130));
    g.drawPolygon(bread1X, bread1Y, 3);
    int[] lettuceX = {
        x - 65, x + 55, x + 5
    };
    int[] lettuceY = {
        y - 20, y - 20, y + 40
    };
    g.setColor(new Color(70, 160, 65));
    g.fillPolygon(lettuceX, lettuceY, 3);
    g.setColor(new Color(225, 55, 45));
    g.fillOval(x - 35, y - 10, 28, 16);
    g.fillOval(x + 5, y - 5, 28, 16);
    g.setColor(new Color(255, 210, 45));
    int[] cheeseX = {
        x - 50, x + 45, x + 5
    };
    int[] cheeseY = {

```

```

        y - 28, y - 28, y + 25
    };
    g.fillPolygon(cheeseX, cheeseY, 3);
    int[] bread2X = {
        x - 80, x + 80, x + 10
    };
    int[] bread2Y = {
        y + 5, y + 5, y + 80
    };
    g.setColor(new Color(230, 180, 90));
    g.fillPolygon(bread2X, bread2Y, 3);
    g.setColor(new Color(160, 105, 40));
    g.drawPolygon(bread1X, bread1Y, 3);
    g.drawPolygon(bread2X, bread2Y, 3);
}
}
public static void main(String[] args) {
    new FoodOrderingBilling();
}
}

```

OUTPUT :

Foodie Express - Online Food Ordering

FOODIE EXPRESS
ONLINE FOOD ORDERING & BILLING SYSTEM



PIZZA
Rs.250



BURGER
Rs.150



SANDWICH
Rs.100

Customer Name

Hema

Food Item 1

Pizza - Rs.250

Quantity

1

Food Item 2

Burger - Rs.150

Quantity

2

Food Item 3

Sandwich - Rs.100

Quantity

1

PLACE ORDER

RESET

YOUR BILL

FOODIE EXPRESS
FINAL BILL

Customer Name : Hema

ITEM	QTY	PRICE	AMOUNT
Pizza	1	250.00	250.00
Burger	2	150.00	300.00
Sandwich	1	100.00	100.00

YOUR BILL

Burger	2	150.00	300.00
Sandwich	1	100.00	100.00

Subtotal

: Rs. 650.00

Discount

: Rs. 0.00

Tax (5%)

: Rs. 32.50

GRAND TOTAL

: Rs. 682.50

THANK YOU! VISIT AGAIN!